765 Final Project Draft

Anish Vallabhaneni, Evan Kania

April 13th 2026

1 Introduction

As we approach the FTQC error regime, our current sampling methods begin to break down at low error. Methods like Monte Carlo sampling become unfeasible as the shots required to capture rare error events become increasingly high and very computationally expensive. For Benchmarking decoders it becomes inaccurate to use only Monte Carlo sampling as it fails to capture rare event errors. This is our motivation for exploring how current sampling methods can capture low error behavior. From the paper Beverland et al. [1] it goes over three techniques aimed to work around this issue. Technique I is normal Monte Carlo, Technique II is importance sampling, and technique III is Markov multi-seeded splitting. Technique one is more proof of inaccuracy of normal Monte Carlo sampling. While the two others implement their own techniques towards improving sampling that in turn improve decoder benchmarking. Our main objective is to implement technique II and III from the paper and compare to technique I. This will allow us to verify the papers validity of these new approaches

For validation of these criteria we should be able to recreate the figures of physical vs logical error , and failure spectrum vs failure wight. Beverland et al. [1] and potentially explore the performance of new decoders.

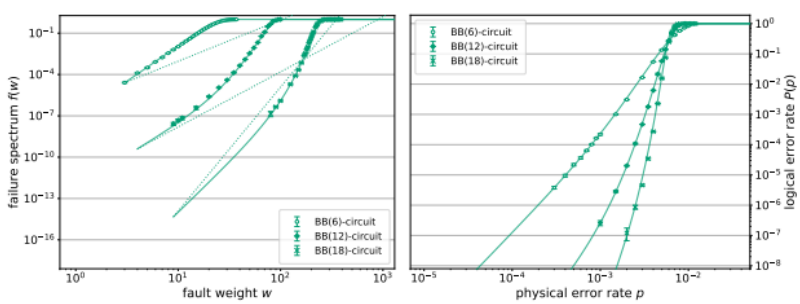

Figure 1: Beverland et al

2 Technique: I

Technique 1 is using normal Monte Carlo sampling to obtain physical vs logical error rate, and failure spectrum vs failure weight graphs. We used distances d=3,5,7 for simplicity(larger distances become too computationally expensive). For larger distances than we chose more shots will be required to acquire meaningful accurate results.

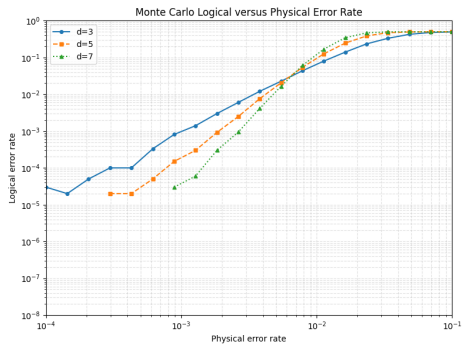

Figure 2: Logical vs Physical Error Rate Monte Carlo Sampling

Figure 2 is extremely close to that of the papers , logical versus physical error rate graphs. The threshold is right where it needs to be just between 10e-1 to 10e-2(good sign). Also after the threshold we can see that a higher distance results in lower logical error. These are the criteria that we wanted to reproduce. the three different curves indicate that at larger distances it suppresses more error. Worth noting that the d=7 line does not go as far left as d=3,5. This was due to not detecting any failures with out shot count at low logical error. This causes the estimator to return zero and hence why we dont see it go as far right.

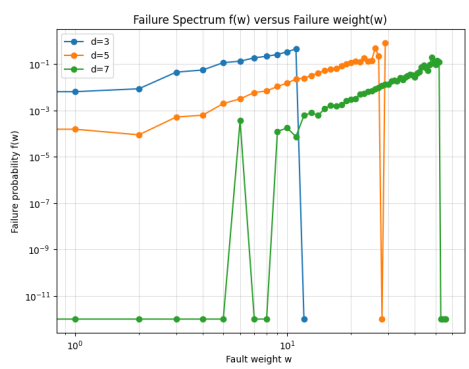

Figure 3: Spectrum Failure vs Failure Weight Monte Carlo Sampling

In figure 3 it roughly recreates figure from paper. Besides for the zero points in lying on the X-axis. These steep peaks occur when it captures zero logical errors for a low physical error. This causes the estimator to be zero which causes it to return zero for d=7 . For far from zero physical error it is zero as well. This is due to that at larger distance it requires more sample shots to be accurate. This makes sense with direct Monte Carlo sampling, as Monte Carlo has bad sampling results at low physical error. This is motivation for technique II and III to improve upon normal Monte Carlo sampling

3 Technique: II

Technique 2 uses importance sampling to bias toward error configurations near the decoding failure threshold, where logical errors are most likely to occur. Each sample is then reweighted to correct for the bias, allowing us to estimate logical error rates without needing to directly observe extremely rare failures. This produces a weighted distribution that approximates the target distribution. We used 5 million shots for technique 1 and 2. In this workflow a semi optimal bias value was determined it by running various bias values handpicking the best figure results. It is worth noting that if we have our bias too low it wont capture relevant information and as well as a too high bias value. This approach is designed such that it speeds up or improves direct Monte Carlo sampling in FTQC error regime

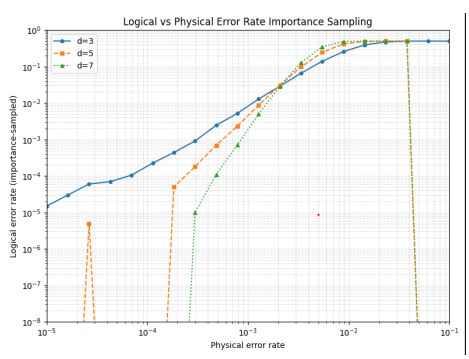

Figure 4: Logical vs Physical Error Rate Importance Sampling

Figure 4 is very close to normal Monte Carlo sampling as seen in Fig 2 technique 1. Larger distances better suppress errors and error threshold is in an acceptable range. The outlier for d=5 must be due to an error in reweighing after first sample. Depending on first distribution some errors can be weighed wrong if not enough shots. Increasing shots helps eliminate "zig-zag" but requires all a longer run time. For d=7 as stated in technique 1, the entries that are on the

zero line are when it doesn't capture any failures and estimator returns zero. This is common for higher distances as seen. d=7 shows a steeper line d=7 line in Fig 2, showing that it suppresses errors faster than direct Monte Carlo approach in technique 1

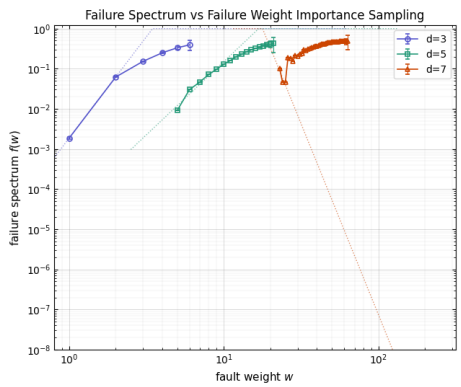

Figure 5: Spectrum Failure vs Failure Weight Importance Sampling

As seen in the Fig 5 above for d=3,5,7 we have smoother failure spectrum's. As physical error rate approaches zero, all distances approach an asymptote which we hope to see. The code gives us three distinct curves, which get smaller as d increases. This is normal as for larger distances it rejects lower weight errors and only accounts for higher ones. This causes d=7 curve to be smaller and have errors that hold more weight(rare event errors). Despite this this the failure spectrum varies smoothly and increases monotonically with w. which are the conditions of a good failure spectrum vs failure weight given in Beverland et. Fig 5 is an notable improvement over technique 1 as is smoother and no outlier data points suggestion that failures were detected. This is evidence that importance sampling obtains sharper results than direct MCS. Even though we obtained sharper results we are still limited by MCS approaches. This is further motivation to implement sampling techniques that work in the fault tolerant error regime. .

4 Technique: III

Technique III describes the Splitting method using metropolis sampling in order to estimate failure rate. The idea of the algorithm is to start with failure rate that can be adequately sampled using monte carlo trials, and then to split into a geometric series of probability ratios towards the desired error probability and to use metropolis sampling to estimate the ratios, using the product of the ratios to estimate the final logical failure rate. [1]

In order to validate our implementation, we created a dataset containing a check matrix, observable matrix, noise model, using decoder-bench[2] to simu-

late asymmetric toric codes with Z noise in order to compare to Figures 7 and 8 from Beverland et al.

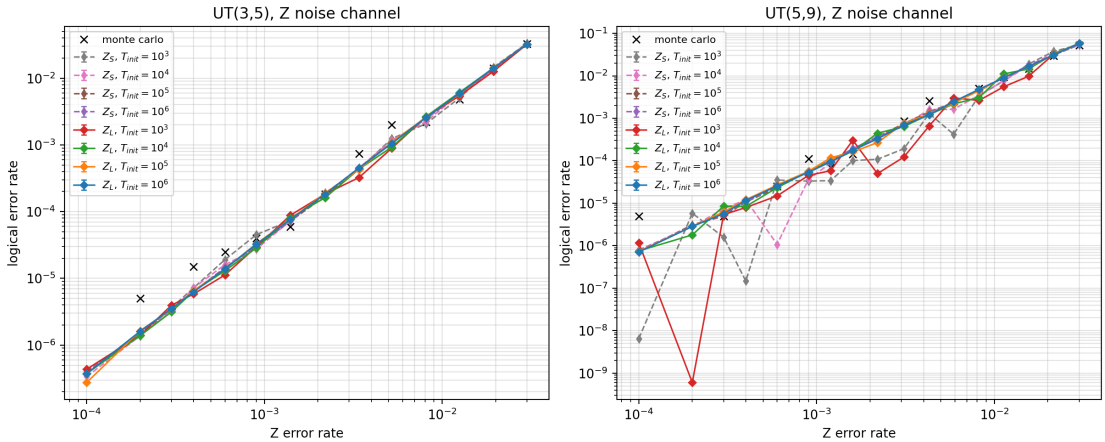

Figure 6: Logical Error rate vs Z error rate for UT(3, 5)/UT(5, 9) Z noise channel with Pymatching as the decoder. Compares the various error models with monte carlo sampling

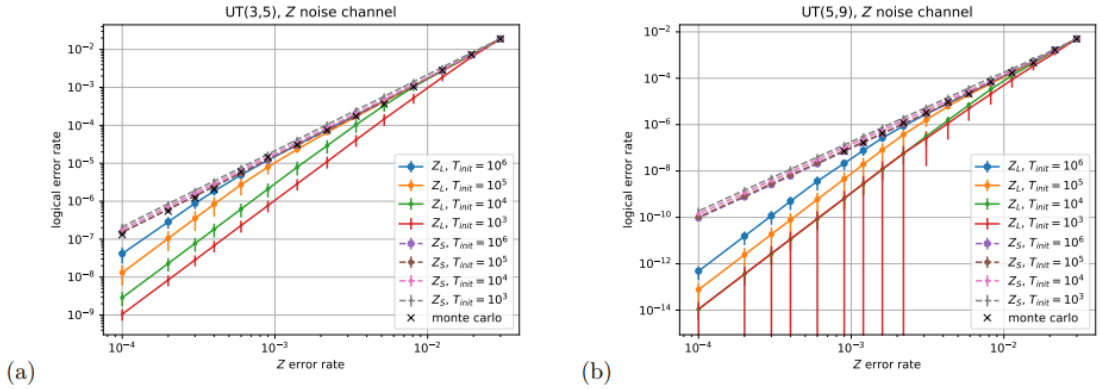

Figure 7: Beverland et al.

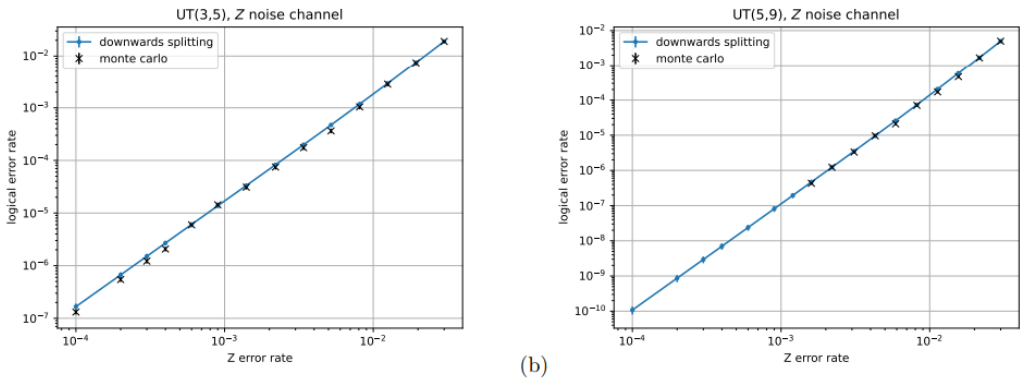

Figure 8: Beverland et al.

While there is clear numerical differences, the point of this figure comparison was to validate the logic of our splitting method. Firstly, the paper did not provide the specific QEC, decoding, or error model details used to generate this data beyond that it is an asymmetric unrotated toric code with a matching decoder and that the initial error rate was p0 = 0.03. The additional details needed in order to create a robust STIM circuit simulation were not provided and could only be assumed. Additionally, the figures from Beverland et al. were produced using fine grain approximation with very high shot counts and repetition, while our figures were a very course approximations that only ran a single shot and did allow for sufficient monte carlo mixing, in order to produce a figure in a reasonable time frame on our personal machines. In fact our shot count, sampling time, and repetitions were all several orders of magnitude smaller than would be expected for rigorous results. We believe these factors sufficiently explain the numerical discrepancies between our figures and the figures from Beverland et al.

However, the exact numerical estimations calculated is not what is needed to validate our algorithm. The important behavior as explained in Beverland et al. is that as T_{init} increases, the approximation from the splitting method should approach the 'true' logical error rate. Keeping in mind the lack of multiple runs, we believe that Figure 6 (a) and Figure 7 (a) show correct behavior of our method. The lower T_{init} values show a wide error margin that converges to the expected error rate. We believe that the performance of our method on the $T_{\text{init}} = 10^6$ (blue lines) estimations show that our method is much closer to the expected logical error rate than even monte carlo sampling (again keeping in mind that our monte carlo sampling was insufficiently mixed though much quicker).

Again keeping in mind that the UT(5, 9) system is a higher code distance and thus a lower error rate system which was poorly approximated by our STIM circuit simulation, we believe that the closer numerical agreement between

Figure 6(a) and Figure 7(a), despite several orders of magnitude more error allowed in our computation, reflects that our code is properly implementing the estimation. This leaves the last remaining step for Technique III as integration with the decoder-bench package.

5 Full Stack Impact

This project is primarily contained to the boundary between the logical and physical circuit compilation layers. This project will provide a way to quantify the performance of various QECC + Decoder combinations in the low error rate regime enabling accurate resource estimation of high performance hardware platforms.

6 Limitations and Future Work

It would be helpful to compare all three techniques using the same hardware restraints to compare the validity of each approach. This would allow us to make stronger claims of technique out performance. Results in technique 1 and 2 are dominated by outlier data points. We could have obtained sharper results by removing the outlier data points. We ultimately kept them to show bare performance of MCS and where it begins to break down.

Showing how MCS performance increases with shot count was not explicitly shown, but explained throughout the paper. We could have ran multiple shot counts to show this correlation instead of assuming. In figure 7 we ran the spitted seed method only once and obtain some nasty looking outliers. Instead doing multiple runs we would expect to see an increase in quality and lessen the outliers in the graph.

References

[1] Michael E. Beverland, Malcolm Carroll, Andrew W. Cross, and Theodore J. Yoder. Fail fast: techniques to probe rare events in quantum error correction, 2025.

[2] Satvik Maurya, Joshua Viszlai, Nithin Raveendran, Poulami Das, and Swamit Tannu. decoder-bench: Benchmarking decoders for quantum error correction, 2025.

7 Appendix: Codebase

All code can be found on the Github: https://github.com/AnishKV1230/low-error-sampling/tree/main

- ‘gen.py’ uses decoder-bench[2] to create the dataset to simulate distance 7 rotated Z memory surface code, which acts as the check matrix, observable matrix, and noise model used for simulations and tests found in ‘my_dataset.h5’
- Files are grouped by technique/use
- Dependencies and version information are listed in ‘requirements.txt’
- Code can be used as an API import if downloaded locally